

Rapport – Tâche 3

Papa Moussa Diabaté & Laurent Polzin

November 21, 2025

Contents

1	Introduction	2
2	GitHub Actions	3
2.1	Modification du workflow	3
2.2	Script Shell pour le score PIT	3
2.3	Mise à jour des artefacts	4
3	Tests et Mockito	5
3.1	Mocks pour tester la suppression de dossier	5
3.2	Mocks pour HttpURLConnection	6
4	Rickroll sur échec des tests	7
5	Conclusion	8

Chapter 1

Introduction

Cette tâche porte sur l'intégration de GitHub Actions, l'utilisation de la librairie Mockito pour les tests unitaires, et la mise en place d'un Rickroll humoristique en cas d'échec des tests.

Chapter 2

GitHub Actions

2.1 Modification du workflow

Nous avons ajouté une action empêchant un `git push` si le score de mutation PIT est inférieur au score précédent. Pour rechercher les scores précédents, nous utilisons :

- `workflow_search: true`
- `workflow_conclusion: success`

Le premier run n'ayant pas d'historique, nous utilisons `continue-on-error: true` pour éviter l'échec automatique.

2.2 Script Shell pour le score PIT

Nous générons le score de mutation puis en extrayons le nombre de mutations tuées grâce au script suivant :

```
mvn clean test-compile -pl core org.pitest:pitest-maven:
mutationCoverage \
    | tee core/core-score

Core_Score=$(grep -A15 Statistics core/core-score | grep Killed \
    | grep -oP '\(\K[0-9]+')
echo "le_score_de_mutation_pour_le_module_core_est_$Core_Score%"

if [ -f core-artefact/Prec_Score ]; then
    Prev_Score=$(cat core-artefact/Prec_Score)
else
    Prev_Score=0
fi

if [ "$Core_Score" -lt "$Prev_Score" ]; then
    echo "Le_score_de_mutation_dans_le_module_core_a_baisse_
        $Prev_Score%-->_$Core_Score%"
    exit 1
fi
```

```
echo "ancien_score:_$Prev_Score%,_nouveau_score:_$Core_Score%"
echo "$Core_Score" > ./Prec_Score
```

S'il détecte une baisse du score, le script génère un `exit 1` pour empêcher le push. Sinon, le score est mis à jour.

Un script identique sera utilisé pour le module `reader-gtfs`.

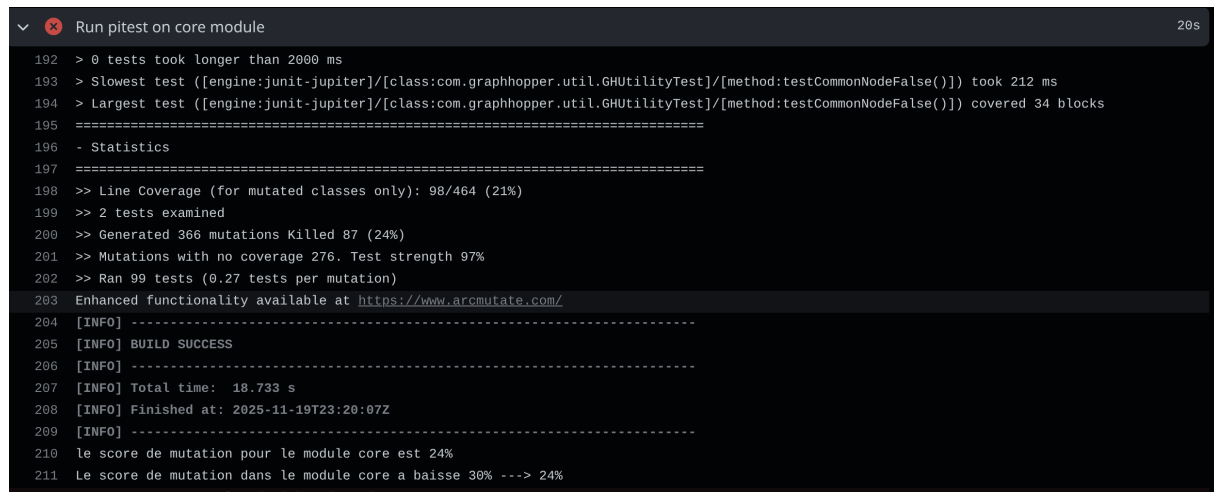
2.3 Mise à jour des artefacts

Exemple pour le module `core` :

```
- name: 'Upload precedent core-score'
  uses: actions/upload-artifact@v4
  with:
    name: core-artefact-${{ matrix.java-version }}
    path: ./Prec_Score
```

Scenario qui fait baisser le score de mutation dans le module `core` en mettant en commentaire (`@Test public void testMerge()`) de la classe :

`core/src/test/java/com/graphhopper/util/ArrayUtilTest.java`



```
Run pitest on core module 20s
192 > 0 tests took longer than 2000 ms
193 > Slowest test ([engine:junit-jupiter]/[class:com.graphhopper.util.GHUtilityTest]/[method:testCommonNodeFalse()]) took 212 ms
194 > Largest test ([engine:junit-jupiter]/[class:com.graphhopper.util.GHUtilityTest]/[method:testCommonNodeFalse()]) covered 34 blocks
195 =====
196 - Statistics
197 =====
198 >> Line Coverage (for mutated classes only): 98/464 (21%)
199 >> 2 tests examined
200 >> Generated 366 mutations Killed 87 (24%)
201 >> Mutations with no coverage 276. Test strength 97%
202 >> Ran 99 tests (0.27 tests per mutation)
203 Enhanced functionality available at https://www.arcmutate.com/
204 [INFO] -----
205 [INFO] BUILD SUCCESS
206 [INFO] -----
207 [INFO] Total time: 18.733 s
208 [INFO] Finished at: 2025-11-19T23:20:07Z
209 [INFO] -----
210 le score de mutation pour le module core est 24%
211 Le score de mutation dans le module core a baisse 36% ---> 24%
```

Figure 2.1: baisse du score de mutation

Chapter 3

Tests et Mockito

3.1 Mocks pour tester la suppression de dossier

Nous avons utilisé Mockito dans `HelperTest.java` (module `web-api`) et `DownloaderTest.java` (module `core`). Les mocks permettent de tester des situations dangereuses, comme la suppression de fichiers, sans impacter le système réel.

```
@InjectMocks
Helper helper;
@Mock File directory;
@Mock File f1;
@Mock File f2;
@Mock File f3;

@Test
public void testRemoveDirectory() {
    when(f1.exists()).thenReturn(true);
    when(f1.delete()).thenReturn(true);

    when(f2.exists()).thenReturn(true);
    when(f2.delete()).thenReturn(true);

    when(f3.exists()).thenReturn(true);
    when(f3.delete()).thenReturn(true);

    when(directory.exists()).thenReturn(true);
    when(directory.delete()).thenReturn(true);

    File[] list = {f1, f2, f3};

    when(directory.isDirectory()).thenReturn(true);
    when(directory.listFiles()).thenReturn(list);

    assertTrue(removeDir(directory));
}
```

3.2 Mocks pour HttpURLConnection

```
@InjectMocks
Downloader downloader;

@Mock
HttpURLConnection connection;

@Test
public void fetchTestNullInputStream(){
    boolean readErrorStreamNoException = false;
    try {
        when(connection.getInputStream()).thenReturn(null);
        assertThrows(IOException.class,
            ()-> downloader.fetch(connection,
                readErrorStreamNoException));
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
}
```

Ce test vérifie que la méthode réagit correctement à une entrée invalide (null).

Chapter 4

Rickroll sur échec des tests

L'objectif est de Rickroll l'utilisateur lorsque les tests échouent.

```
- name: Build ${ matrix.java-version }
  run: mvn -B clean test

- name: Rick Roll On Fail
  if: failure()
  run: |
    echo "![RickRoll](https://media.giphy.com/media/Vuw9m5wXviFIQ/giphy.gif)"
    cat .github/workflows/rickroll.txt
```

Le fichier `rickroll.txt` contient un ASCII Art de Rick Astley, affiché dans les logs GitHub Actions.

Echec du test `empty` de la classe:

`./web-api/src/test/java/com/graphhopper/util/PMapTest.java`

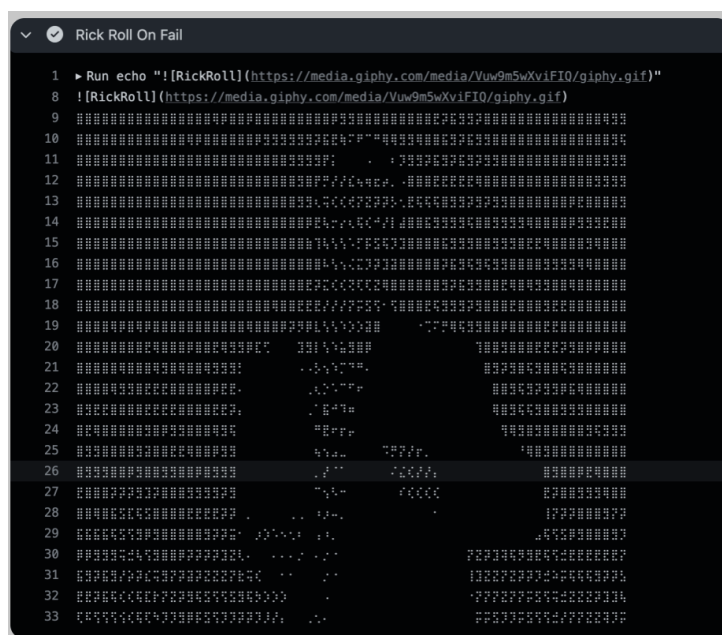


Figure 4.1: echec du test (`public void empty()`)

Chapter 5

Conclusion

Cette tâche nous a permis d'intégrer un workflow avancé avec vérification automatique de la qualité des tests, d'utiliser efficacement Mockito et de créer un Rickroll humoristique via GitHub Actions. Un travail complet mêlant automatisation, tests unitaires et créativité.